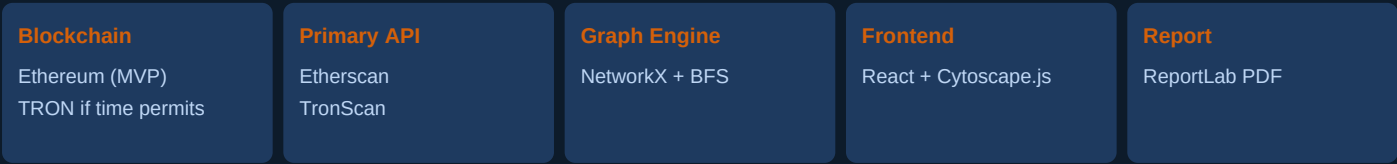


CryptoTrace I4C

Practical Prototype Implementation & Team Workflow

SIH 2026 • Problem Statement SIH26183 • Ministry of Home Affairs / I4C

INTERNAL BUILD GUIDE • 3-DAY HACKATHON • 6-MEMBER TEAM



CORE PIPELINE



TEAM ROLES



WE DO:

- | | | | |
|-----------------------------|-----------------------|-------------------|----------------|
| ✓ Public blockchain tracing | ✓ VASP label matching | ✓ Fund-flow graph | ✓ Evidence PDF |
|-----------------------------|-----------------------|-------------------|----------------|

WE DO NOT:

- | | | | |
|-----------------------|-------------------------|----------------|-------------------|
| ✗ Bank/UPI/KYC access | ✗ Identify real persons | ✗ Freeze funds | ✗ Legal decisions |
|-----------------------|-------------------------|----------------|-------------------|

Table of Contents

- [§1 What CryptoTrace Does — The Simple Explanation](#)
- [§2 Real-World Context — How the Investigator Uses It](#)
- [§3 Complete Software Workflow — Step by Step](#)
- [§4 Data Sources & Field Definitions](#)
- [§5 Fund-Flow Logic & Attribution Method](#)
- [§6 Team Work Distribution — 6 Members](#)
- [§7 How All Modules Connect — Integration Contract](#)
- [§8 Technology Stack](#)
- [§9 3-Day Build Plan](#)
- [§10 Demo Workflow — What the Judge Sees](#)
- [§11 Handling Multiple Cases](#)
- [§12 AI Positioning — What Is & Isn't AI](#)
- [§13 Build Checklist — Physical Deliverables](#)
- [§14 Debugging & Integration Order](#)
- [§15 Final Team Summary — One Page](#)

§1 What CryptoTrace Does — The Simple Explanation

A cybercrime investigator receives a suspect cryptocurrency wallet address from a fraud victim. Manually following every subsequent transaction across block explorer websites is slow, error-prone, and produces no structured evidence. CryptoTrace automates that repetitive work.

Core Idea in One Sentence

Reported fraud → investigator obtains suspect wallet → CryptoTrace → blockchain transaction data → normalisation → graph construction → controlled multi-hop tracing → VASP/address matching → evidence → investigation report.

What CryptoTrace receives as input

- One or more suspect cryptocurrency wallet addresses (provided by investigator)
- The blockchain network (Ethereum for MVP; TRON if time permits)
- Optional: case name, date range, maximum hop depth (default 3)

What CryptoTrace produces as output

- Interactive transaction graph — nodes are wallets, edges are transactions
- Fund-flow path with hop count, amounts, and timestamps
- VASP identification where evidence exists (exchange name, confidence, source)
- Evidence table — transaction hashes, amounts, explorer URLs
- PDF investigation report — structured, downloadable, evidence-backed

What makes it different from manually using Etherscan

Manual Explorer Approach	CryptoTrace Approach
Investigator opens many browser tabs	One wallet input triggers full pipeline
Manually follows each hop	Automated BFS traversal up to N hops
Manually checks if destinations are exchanges	Automatic VASP label lookup with confidence
No structured output	Structured evidence JSON + PDF report
Hours per case	Minutes per case (MVP scale)

Hard boundary: CryptoTrace does NOT identify real persons behind exchange accounts, does NOT access bank/UPI/NPCI/private KYC data, does NOT freeze funds automatically, and does NOT make legal decisions.

§2 Real-World Context — How the Investigator Uses It

Understanding exactly where CryptoTrace fits in the real investigation workflow prevents the team from over-claiming capabilities.

Real Investigation Flow

Step	Actor	What Happens	CryptoTrace Role
1	Victim	Reports fraud — provides details, possibly a wallet address	Not involved
2	Bank/PSP	May investigate payment rail; may share relevant info with investigator	Not involved
3	Investigator	Identifies / obtains the suspect crypto wallet address	Not involved
4	CryptoTrace	Starts here — wallet address entered into the dashboard	STARTS HERE
5	CryptoTrace	Fetches public blockchain data, builds graph, traces hops	Core pipeline

6	CryptoTrace	Matches destination addresses against VASP label database	ASP matching
7	CryptoTrace	Generates evidence JSON + PDF report	Output
8	Investigator	Uses report to send legal/compliance request to identified VASP	Not involved
9	VASP	Provides KYC identity of account holder under legal process	Not involved

Key Clarification: CryptoTrace does NOT receive private bank or UPI data directly. It does NOT have access to NCRP, NPCI, Paytm, or any government systems. It does NOT automatically freeze funds within 24 hours. It starts ONLY when a crypto wallet address is already available to the investigator and operates exclusively on public blockchain data.

§3 Complete Software Workflow — Step by Step

This section describes every step of the software pipeline from the investigator's input to the final PDF report. Every member must understand the full flow, not just their own step.

Step 1 — Investigator Input (React Frontend)

Investigator enters: case name, blockchain (Ethereum), 1–5 suspect wallet addresses, date range, max hops (default 3). Validated in the browser before submission.

Input	Wallet addresses + case metadata
Output	HTTP POST /trace JSON payload
Owner	M3 (Frontend)

Step 2 — Backend Validation (FastAPI / Python)

FastAPI validates: correct Ethereum address format (0x + 40 hex characters), non-empty inputs, hop limit ≤ 3. Returns HTTP 422 with message on invalid input.

Input	Raw POST request
Output	Validated parameters or 422 error
Owner	M2 (Backend)

Step 3 — Blockchain Data Fetch (Etherscan API / JSON cache)

For each wallet, call Etherscan API: normal ETH transactions + ERC-20 token transfers. If live call fails or is slow, load from pre-cached JSON file. Apply date-range filter.

Input	Wallet address + date range
Output	Raw Etherscan JSON response
Owner	M1 (Blockchain)

Step 4 — Transaction Normalisation (Python / Pydantic)

Convert Etherscan response to the unified internal schema: tx_hash, from, to, amount (ETH or token), asset, timestamp (UTC ISO 8601), block, chain, source_url.

Input	Raw Etherscan JSON
Output	Normalised list of transaction dicts
Owner	M1 (Blockchain)

Step 5 — Graph Construction (NetworkX)

Create a directed graph (DiGraph): nodes = unique wallet addresses, directed edges = transactions with attributes (amount, asset, timestamp, tx_hash). Handles multiple suspect wallets in one case.

Input	Normalised transaction list
Output	NetworkX DiGraph object
Owner	M2 (Backend/Graph)

Step 6 — BFS Multi-Hop Traversal (NetworkX BFS)

Starting from each suspect wallet, perform Breadth-First Search up to max_hops (default 3). Collect all reachable nodes and edges. Stop early at VASP-matched nodes.

Input	DiGraph + suspect wallets + max_hops
Output	Subgraph / path list covering all hops
Owner	M2 (Backend/Graph)

Step 7 — Fund Attribution (FIFO) (Python (pure arithmetic))

For each wallet in traversal order, track suspected_balance. traceable = min(suspected_balance, outgoing_amount). Tags each edge with traceable_amount and unclassified_amount. See §5 for full algorithm.

Input	Path list + transaction amounts
Output	Edges annotated with traceable / unclassified amounts
Owner	M2 (Backend/Graph)

Step 8 — VASP Matching (Python dict lookup)

For each destination wallet, look up address in vasp_labels.json. Exact match: assign entity, type, confidence=HIGH. No match: confidence=UNKNOWN. Never guess from behaviour alone.

Input	Address list
Output	VASP label record OR UNKNOWN
Owner	M2 (Backend/Graph)

Step 9 — Evidence Assembly (Python)

Combine: transaction hashes, from/to addresses, amounts, traceable amounts, VASP labels, timestamps, block numbers, Etherscan explorer URLs. Structure as evidence[] list.

Input	All data from Steps 3-8
Output	Structured evidence JSON
Owner	M4 (Integration)

Step 10 — PDF Report Generation (ReportLab)

ReportLab builds PDF: case info header, fund-flow path diagram, attribution table, VASP matches with confidence, evidence hashes, limitation statement.

Input	Evidence JSON
Output	PDF file saved to disk, download URL returned
Owner	M4 (Integration)

Step 11 — API Response (FastAPI)

FastAPI returns complete `/trace` JSON: case, wallets, graph nodes/edges, paths, suspected amounts, VASP matches, evidence list, report download URL, `data_source` (live/cached).

Input	Evidence JSON + report URL
Output	HTTP 200 JSON response
Owner	M2 (Backend)

Step 12 — Frontend Rendering (React + Cytoscape.js)

React renders: interactive Cytoscape graph (nodes coloured by role), fund-flow path panel, evidence table, confidence badges, PDF download button.

Input	JSON response from <code>/trace</code>
Output	Interactive investigation dashboard
Owner	M3 (Frontend)

§4 Data Sources & Field Definitions

A. Blockchain Transaction Data

All blockchain data comes from public APIs. No private or government data is required for the MVP.

Field	Etherscan Key	Notes
chain	(hardcoded: ETH)	Set at input; only ETH in MVP
tx_hash	hash	Primary evidence reference — unique per transaction
from	from	Source wallet address
to	to	Destination wallet address
amount	value ÷ 10 ¹⁸	ETH amount. For ERC-20: value ÷ 10 ^{decimals}
asset	ETH or tokenSymbol	Distinguishes ETH from token transfers
timestamp	timeStamp → ISO 8601	Convert Unix epoch to UTC datetime string
block	blockNumber	Blockchain evidence context
source_url	Built from hash	<code>etherscan.io/tx/{hash}</code> — verifiable link for report

Etherscan endpoints used:

Normal ETH: `?module=account&action=txlist&address=...&sort=asc`

ERC-20 tokens: `?module=account&action=tokenTx&address=...&sort=asc`

API key: free registration at etherscan.io — 5 calls/sec, 100k calls/day.

B. VASP Label Data

VASP identification relies on exact address matching against a curated JSON dataset of known exchange/custodian wallet addresses. This dataset is built manually before the hackathon. **Quality matters more than quantity** — 30–40 verified records are sufficient for the demo.

Field	Example	Notes
address	0x3f5ce5f...	Ethereum address (lowercase) — exact match key
chain	ETH	Network identifier
entity	Example Exchange	Exchange or service name
type	exchange	exchange custodian defi mixer unknown

source	Etherscan label	Where this attribution comes from
source_url	https://etherscan.io/...	Verifiable source link
source_date	2026-08-01	Date the label was verified
confidence	HIGH	HIGH MEDIUM (never guess)

How to collect VASP records: (1) Etherscan Labels page — major exchange hot wallet deposit addresses. (2) Cross-verify each address in a second source before adding. (3) Record the source URL and date for every entry. (4) Do NOT add addresses from unverified forum posts.

C. Evidence Data

Evidence is the combined output that goes into both the frontend display and the PDF report. Each evidence record is tied to a specific transaction hash — the judge can independently verify every record.

- hop — which hop number in the trace (1, 2, 3)
- from / to — wallet addresses involved
- amount — total amount sent in this transaction
- traceable_amount — portion attributed to suspected funds (FIFO method)
- unclassified_amount — remainder not attributed
- tx_hash — the blockchain transaction identifier
- timestamp — when the transaction occurred (UTC)
- block — block number for blockchain context
- vasp — entity name if matched, 'UNKNOWN' otherwise
- confidence — HIGH | UNKNOWN
- explorer_url — full Etherscan link to the transaction

§5 Fund-Flow Logic & Attribution Method

This section explains the most important technical decision in the project. The FIFO (First-In First-Out) fund attribution algorithm must be clearly understood by every team member — especially M2 who implements it, and M5/M6 who must explain it to judges.

The Core Problem

When Wallet B receives funds from a suspected fraudster (Wallet A) AND also holds other money from unrelated sources, we cannot claim the ENTIRE balance of Wallet B is fraud money. The system must track only the specific suspected portion.

FIFO Algorithm — How It Works

```

State per wallet: suspected_balance = 0.0
On incoming transaction to wallet W:
IF source is a suspect wallet OR source has suspected_balance > 0:
    suspected_balance[W] += incoming_amount
ELSE: normal_balance[W] += incoming_amount

On outgoing transaction from wallet W (sorted oldest first):
traceable = min(suspected_balance[W], outgoing_amount)
unclassified = outgoing_amount - traceable
suspected_balance[W] -= traceable
Tag edge: traceable_amount = traceable, unclassified_amount = unclassified

```

Worked Example

Step	Transaction	Traceable	Unclassified	suspected_balance
Setup	Wallet A has 100 ETH pre-existing (normal) Victim → Wallet A: 10 ETH (suspected)			A = 10 ETH

Hop 1	Wallet A → Wallet B: 20 ETH sent	$\min(10, 20) = 10 \text{ ETH}$	$20 - 10 = 10 \text{ ETH}$	A = 0, B = 10 ETH
Hop 2	Wallet B → Wallet C: 5 ETH sent	$\min(10, 5) = 5 \text{ ETH}$	0 ETH	B = 5 ETH, C = 5 ETH

Key Point for Judges: The system does NOT claim the outgoing 20 ETH is all fraud money. It calculates the maximum attributable amount under a conservative FIFO rule and clearly labels the rest as unclassified. This is an investigative lead, not a verdict.

Important Limitations — State These to Judges

- Blockchain does not tag individual coins. This is transaction-level heuristic attribution, not legal proof.
- FIFO ordering assumes time-sorted transactions; may differ from actual wallet management.
- A wallet with large existing balance reduces the proportion of traceable amount — this is intentional and conservative.
- Result is an investigative lead. Legal action requires human review and appropriate court process.

§6 Team Work Distribution — 6 Members

Every member must understand the complete system. Nobody should know only their own module. M5 and M6 are not purely presentation members — they assist with testing and documentation when free.

M1 — Blockchain / Data Engineer

Tech: Python, FastAPI, Etherscan API, TronScan (optional), JSON, Pydantic

Responsibilities

- Etherscan API integration (normal transactions + ERC-20 token transfers)
- TronScan API if time permits after ETH is fully working
- API response handling, rate-limit retry (0.25s delay between calls)
- Transaction normalisation — convert to unified internal schema
- JSON cache files for demo reliability and API fallback
- Explain data sources clearly to team and judges

Files / Modules	services/fetcher.py, services/normalizer.py, models/transaction.py, cache/eth_cache.json
Receives from	Wallet address + date range from backend validation
Delivers to	Normalised transaction list (JSON) → M2 and M4

Day-by-Day Plan

- Day 1: Etherscan API working for one wallet. Raw response → normalised schema. Verify with test wallet.
- Day 2: Multi-wallet dedup by tx_hash. Date filtering. JSON cache fallback. Error handling.
- Day 3: Integration test with M2 output. Harden cache. TronScan if ETH fully stable.

M2 — Backend + Graph Engineer

Tech: Python, FastAPI, NetworkX, JSON/SQLite, Pydantic

Responsibilities

- FastAPI backend — main.py, routes, validation, /trace endpoint
- NetworkX DiGraph construction from normalised transaction list
- BFS multi-hop traversal (default max 3 hops)
- FIFO fund attribution algorithm — test with at least 5 edge cases
- VASP exact-address matching from vasp_labels.json
- Backend API response — complete /trace JSON schema (§7)

Files / Modules	main.py, api/routes.py, services/graph_builder.py, services/attribution.py, services/vasp_matcher.py
-----------------	--

Receives from	Normalised transaction list from M1
Delivers to	Graph JSON + evidence JSON → M3 (frontend) and M4 (PDF)

Day-by-Day Plan

- Day 1: Build DiGraph + BFS with hardcoded test data (no need to wait for M1). Test FIFO with \$5 example.
- Day 2: Connect to M1 live data. Full BFS on real wallet. FIFO attribution on real transactions. VASP lookup.
- Day 3: Edge case testing (no outflow, cycles, all UNKNOWN). Fix bugs. Final integration.

M3 — Frontend Engineer

Tech: React, Vite, TypeScript, Cytoscape.js, Tailwind CSS (optional)

Responsibilities

- React dashboard — case input form, chain selection, Trace button
- Loading states (progress bar: Fetching → Normalising → Building graph → Matching)
- Error state handling (invalid address, API failure)
- Cytoscape.js interactive graph — colour-coded nodes (red=suspect, yellow=intermediary, green=VASP)
- Transaction/evidence panel — evidence table with explorer links
- VASP confidence display — badges (HIGH = green, UNKNOWN = grey)
- PDF download button

Files / Modules	src/components/CaseForm.tsx, GraphView.tsx, EvidenceTable.tsx, ConfidenceBadge.tsx
Receives from	/trace JSON response from FastAPI (\$7 schema)
Delivers to	Working investigation dashboard for demo

Day-by-Day Plan

- Day 1: Build complete UI with MOCK JSON data matching \$7 schema. Do NOT wait for backend.
- Day 2: Connect to live /trace endpoint. Render real Cytoscape graph. Show real evidence table.
- Day 3: Loading/error states polished. PDF download button. Demo rehearsal. Backup screenshots.

M4 — Integration + Evidence + Testing Engineer

Tech: Python, ReportLab, JSON/SQLite, pytest, Git/GitHub

Responsibilities

- Connect frontend and backend — ensure /trace endpoint and UI speak the same JSON
- VASP dataset integration — final vasp_labels.json wired into backend
- Evidence structure — ensure all fields are populated correctly
- ReportLab PDF generation — case info, fund-flow, attribution table, VASP, evidence, limitations
- Curate 30-40 verified VASP records in vasp_labels.json with source + date
- Cache/fallback — eth_cache.json tested and working for demo reliability
- End-to-end integration testing — full flow working 5 times consecutively
- Test cases — write unit tests for FIFO attribution (at minimum 5 edge cases)

Files / Modules	services/report_generator.py, services/evidence_builder.py, data/vasp_labels.json, demo/eth_cache.json
Receives from	Evidence JSON from M2; case + wallet records from M1
Delivers to	PDF download endpoint /reports/{case_id}.pdf; working tested demo

Day-by-Day Plan

- Day 1: Curate 30+ VASP records. Design PDF layout in ReportLab with sample data. Save demo cache.
- Day 2: Wire evidence JSON into PDF template. Generate complete PDF. Expose GET /reports/{case_id}.
- Day 3: Full integration test. PDF download from UI. Demo rehearsal. Fallback cache verified.

M5 — Presentation + Q&A; (Primary)

Tech: Presentation tools; complete understanding of all technical components

Responsibilities

- PPT/story consistency with technical implementation (no over-claiming)
- Problem explanation — clear narrative for judges in 60-90 seconds
- Solution explanation — demonstrate the complete investigation flow
- Demo narration — explain what is happening on screen at each step
- Impact and limitations — honest, defensible statements
- Judge Q&A; preparation — know the top 10 judge questions and answers
- When free: assist M4 with VASP dataset curation and testing
- Must understand complete technical workflow to answer judge questions

Files / Modules	PPT, demo script, judge Q&A document
Receives from	Technical details from all members
Delivers to	Clear, defensible demo narrative + judge Q&A responses

Day-by-Day Plan

- Day 1: Draft problem/solution slides. Understand M1 data flow completely.
- Day 2: Draft demo script. Review M2 graph output. Help M4 with VASP curation.
- Day 3: Full demo rehearsal. Time the demo. Prepare all judge Q&A; answers.

M6 — Presentation + Q&A; (Support)

Tech: Documentation tools; complete understanding of all technical components

Responsibilities

- Support M5 on all presentation and demo responsibilities
- Technical backup — can explain any module if a judge asks
- Documentation — README, setup instructions, screenshots
- When free: assist with testing (run the full pipeline repeatedly)
- Video backup recording — record a clean demo run as insurance
- Timing measurements — record actual retrieval, graph, and report times

Files / Modules	README.md, screenshots, demo video backup
Receives from	Technical details from all members
Delivers to	Documentation, screenshots, video backup, measured timings

Day-by-Day Plan

- Day 1: Set up GitHub repo, README structure. Understand all module interfaces.
- Day 2: Document integration contract. Take screenshots of each working step.
- Day 3: Record video backup. Write limitation statements. Final timing measurements.

§7 How All Modules Connect — Integration Contract

The integration contract defines the common JSON schema that all members must agree on **before Day 1 coding starts**. M3 builds the UI against this schema using mock data. M1/M2/M4 fill in real values. This prevents integration failures on Day 3.

Module Connection Diagram

M1 — Blockchain/Data	↓
Etherscan API → Raw JSON → Normalised Transactions (tx_hash, from, to, amount, timestamp...)	↓

M2 – Backend + Graph	↓
FastAPI → NetworkX DiGraph → BFS Traversal → FIFO Attribution → VASP Lookup → /trace JSON	↓
M3 – Frontend	↔
React + Cytoscape.js Dashboard → Displays graph, evidence, VASP, confidence badges	↓
M4 – Integration + PDF	↓
Evidence Assembly → ReportLab PDF → /reports/{case_id}.pdf	
M5/M6 – Presentation & Q&A	■ All components

Common JSON Schema — /trace Request

```
POST /trace
{
  "case_id": "CASE-001",
  "case_name": "Online Fraud Case 001",
  "chain": "ETH",
  "wallets": ["0xAAA...", "0xBBB..."],
  "max_hops": 3,
  "start_date": "2024-01-01",
  "end_date": "2024-12-31"
}
```

Common JSON Schema — /trace Response (200 OK)

```
{
  "case_id": "CASE-001",
  "status": "complete",
  "wallets": [{"address": "0xAAA...", "role": "suspect"}],
  "summary": {
    "total_transactions": 248,
    "total_suspected_eth": 10.0,
    "hops_traced": 3,
    "vasp_matches": 1
  },
  "graph": {
    "nodes": [
      {"id": "0xAAA...", "role": "suspect", "suspected_balance": 0.0},
      {"id": "0xBBB...", "role": "intermediary", "suspected_balance": 5.0},
      {"id": "0xEXCH...", "role": "vasp", "suspected_balance": 0.0}
    ],
    "edges": [
      {"from": "0xAAA...", "to": "0xBBB...", "amount": 20.0, "asset": "ETH",
        "traceable_amount": 10.0, "unclassified_amount": 10.0,
        "tx_hash": "0x123...", "timestamp": "2024-06-01T10:00:00Z",
        "block": 19000000, "hop": 1}
    ]
  },
  "vasp_matches": [
    {"address": "0xEXCH...", "entity": "Example Exchange", "type": "exchange",
      "confidence": "HIGH", "source": "Etherscan label",
      "source_url": "https://etherscan.io/..."}
  ],
  "evidence": [
    {"hop": 1, "from": "0xAAA...", "to": "0xBBB...", "amount": 20.0,
      "asset": "ETH", "traceable_amount": 10.0, "unclassified_amount": 10.0,
      "tx_hash": "0x123...", "timestamp": "2024-06-01T10:00:00Z",
      "block": 19000000, "vasp": "UNKNOWN", "confidence": "UNKNOWN",
      "explorer_url": "https://etherscan.io/tx/0x123..."}
  ],
  "report_url": "/reports/CASE-001.pdf",
  "data_source": "live"
}
```

Rule: Lock this schema on Day 1 morning. All members sign off before coding. M3 builds the entire UI using mock data matching this shape. Schema changes must be agreed by ALL members — not made unilaterally.

§8 Technology Stack

Use the minimum technology that can demonstrate the complete flow. Do not add technologies to make the architecture look impressive. A smaller working system beats a large architecture that cannot be demonstrated.

Layer	Technology	Decision	Reason
Frontend	React + Vite + TypeScript	KEEP	Fast dev server; TypeScript catches errors early
Graph Visualisation	Cytoscape.js	KEEP	Best free interactive graph library for React
Styling	Tailwind CSS or plain CSS	KEEP	Fast UI iteration; avoid time on design systems
Backend	Python + FastAPI	KEEP	Fastest Python API framework; async; easy testing
Graph Analysis	NetworkX	KEEP	BFS + DiGraph is exactly what we need
Blockchain (ETH)	Etherscan API	KEEP	Free tier (5 calls/sec); sufficient for demo depth
Blockchain (TRON)	TronScan API	DEFER	Add ONLY after full ETH flow works end-to-end
Data Storage	JSON initially / SQLite if needed	KEEP	Simple, transparent, easy to ship
PDF Generation	ReportLab	KEEP	Programmatic PDF; no browser dependency
Version Control	Git + GitHub	KEEP	Essential for 6-member coordination
AI / LLM	Claude/Gemini API	OPTIONAL	Narration only; after deterministic pipeline is stable
Database (optional)	MongoDB	MVP PLAN	Schema-flexible; local instance; no cloud required
Docker / Kubernetes	—	REMOVE	Unnecessary complexity for a local hackathon demo
BSC / other chains	—	REMOVE	Scope creep; ETH alone is sufficient
ML pipelines	—	REMOVE	Not required for deterministic FIFO approach

§9 3-Day Build Plan

This plan is realistic and based on the existing scope decisions. Do not add Day 4 or Day 5 features. The stop rule: if any new feature is not essential to demonstrating the core flow, cut it immediately and move to Future Scope.

DAY 1 Foundation + Real Blockchain Data	
Member	Task
M1	Set up FastAPI project. Integrate Etherscan API for one wallet. Normalise response to common schema. Test: enter wallet → normalised
M2	Build DiGraph + BFS with hardcoded test transaction list (do not wait for M1). Implement FIFO attribution. Test with \$5 worked example.
M4	Manually fetch one public demo case from Etherscan. Save as eth_cache.json. Curate 30+ VASP records. Start PDF template layout in f
M3	Build complete React UI with MOCK JSON (the §7 schema). CaseForm, GraphView placeholder, EvidenceTable, ConfidenceBadge. Do
M5/M6	Draft problem/solution slides. Set up GitHub repo and README. Understand all module interfaces.

End of DAY 1: M1 can fetch+store live ETH transactions. M2 attribution works on test data. M4 has demo cache + VASP file. M3 UI renders mock data.

DAY 2 Graph + Fund Attribution + VASP + Integration	
Member	Task

M1	Add multi-wallet dedup by tx_hash. Add date filter. Test cache fallback. Expose POST /trace endpoint returning raw transactions.
M2	Connect to M1 data. Run full BFS on live transactions. Apply FIFO attribution to real wallet trace. Integrate vasp_labels.json from M4. Re
M4	Wire evidence JSON into ReportLab PDF. Generate complete PDF with all sections. Expose GET /reports/{case_id} endpoint.
M3	Connect to live /trace endpoint. Render real Cytoscape graph. Show colour-coded nodes. Display real evidence table. Integrate confiden
M5/M6	Draft demo script. Take screenshots of each working step. Document the integration contract.

End of DAY 2: Full investigation flow works: wallet input → graph with real data → VASP matched node → evidence table. PDF generates. UI shows real results.

DAY 3 Evidence + PDF + Testing + Demo Hardening

Member	Task
M1	Harden error handling (invalid address, API 429, timeout). Add /health endpoint. Final integration test. TronScan only if ETH fully stable.
M2	Edge-case testing: wallet with no outflow, cycle detection, all UNKNOWN VASPs, large hop count. Fix all bugs found.
M4	Final PDF polish: limitation statement, confidence explanation, investigator guidance section. Test PDF download from UI. 5 consecutive
M3	Loading state, error messages, PDF download button polished. Full demo rehearsal. Prepare backup screenshots and cached demo sce
M5/M6	Full demo rehearsal. Time the demo (target 90 seconds). Prepare all judge Q&A answers. Record video backup.

End of DAY 3: Demo runs reliably 5 consecutive times from cached data. PDF downloads correctly. All judge Q&As; prepared. Video backup recorded.

NOT on Day 2 or Day 3: TRON support (until ETH fully stable), LLM narration, performance optimisation, Docker, new features not in the core flow.

§10 Demo Workflow — What the Judge Sees

The judge should understand the value within the first 60–90 seconds. Pre-cache the demo case. Run it 5 times before the presentation. Always have a video backup.

Pre-Demo Preparation (Before Presenting)

- Find a publicly documented Ethereum fraud/scam case with verifiable transaction trail (academic papers, public blockchain security reports, Etherscan-documented scam addresses)
- Manually verify the path: scammer wallet → 1–2 intermediary wallets → known exchange deposit address
- Save the Etherscan API responses as eth_cache.json. Ensure the exchange destination address is in vasp_labels.json
- Run the full pipeline on cached data 5 times before the presentation
- Record a backup video of the complete demo flow running correctly

Live Demo Steps — Judge View

#	Step	Type	Description
1	Open dashboard	LIVE	Open CryptoTrace. Enter case name 'Demo Case 001'. Select Ethereum.
2	Enter wallet address	LIVE	Paste the documented suspect wallet address. Click Trace.
3	Fetch + progress bar	LIVE/CACHED	Progress shows: Fetching → Normalising → Building graph → Matching. API from cache if needed.
4	Normalised transactions	LIVE/CACHED	Show the transaction table: tx_hash, from, to, amount, timestamp. These are real blockchain records.
5	Graph appears	LIVE/CACHED	Cytoscape graph: red node (suspect) → yellow node (intermediary) → green node (VASP/exchange).
6	Click edge A→B	LIVE/CACHED	Edge panel: Total sent 20 ETH, Traceable to victim 10 ETH, Unclassified 10 ETH.

7	VASP match	LIVE/CACHED	Exchange node highlighted GREEN: 'Example Exchange • HIGH confidence • Source: Etherscan Lab
8	Evidence table	LIVE/CACHED	Scroll to evidence: tx_hash, from, to, amount, traceable, timestamp, Etherscan link, confidence. Every
9	Open explorer	LIVE	Click Etherscan link for one transaction. Judge sees the same hash on etherscan.io.
10	Generate PDF	LIVE	Click Generate Report. After 2-3 seconds, PDF downloads. Shows case info, fund-flow, evidence, lim
11	UNKNOWN case (optional)	SYNTHETIC	Show what happens when VASP is unknown: UNKNOWN badge, still shows transaction path and evi

What to Clearly Distinguish to Judges

LIVE (actual real-time)	SYNTHETIC (clearly labelled)	NOT AVAILABLE
Public blockchain data retrieval Etherscan API (or cached) VASP label lookup Explorer link verification PDF generation	Demo wallet scenario if no public case is available — clearly labelled 'Demo Scenario' in the UI	Bank/UPI/NPCI data Private KYC access Real person identity Automatic fund freezing Official NCRP/I4C integration

Never fabricate: wallet addresses, transaction hashes, screenshots, performance numbers, or VASP attributions that are not in your vasp_labels.json.

§11 Handling Multiple Cases

The same application can handle multiple investigation cases independently. Each case has its own case_id, which ties all its transactions, graph, evidence, and report together.

Case ID	Input Wallet	Output
CASE-001	0xAAA... (Ethereum)	Report_CASE-001.pdf — fund-flow, VASP match, evidence
CASE-002	0xBBB... (Ethereum)	Report_CASE-002.pdf — different wallet, independent trace
CASE-003	T9abc... (TRON, if built)	Report_CASE-003.pdf — TRON trace, separate evidence

Each investigation is stored separately by case_id. The investigator can run multiple cases and generate independent PDF reports for each one. The MVP scale is demonstration-level — a small number of cases for the hackathon demo.

Realistic Scale Path

- MVP (now): Ethereum, 1–5 wallets per case, demo-scale data, local storage
- Phase 2: More chains (TRON, BSC), larger VASP dataset, persistent case database
- Phase 3: Stronger infrastructure, authorised institutional data sources, formal NCRP/I4C integration

Do not claim unlimited production scale for the MVP.

§12 AI Positioning — What Is & Isn't AI

Do not describe CryptoTrace as 'AI-powered.' The core pipeline is deterministic and that is a strength, not a weakness. Deterministic logic is faster, safer, and more defensible to judges.

Component	AI?	What It Actually Is
Etherscan API fetch	NO	Deterministic HTTP API call
Transaction normalisation	NO	Pure Python field mapping
NetworkX graph construction	NO	Standard graph library
BFS traversal	NO	Standard graph algorithm
FIFO fund attribution	NO	Pure arithmetic — fully explainable

VASP exact-address match	NO	Dictionary/JSON lookup
Confidence rules	NO	Deterministic rule evaluation
PDF report generation	NO	ReportLab programmatic rendering
Plain-English summary (optional)	OPTIONAL	LLM receives verified JSON, converts to readable text

If AI (LLM) is used at all, the contract is:

- LLM receives only verified, structured JSON output from the deterministic pipeline
- LLM may convert the verified JSON into a readable investigation summary
- LLM must NOT invent wallet addresses, transaction hashes, VASP identities, person names, or evidence
- LLM is an optional layer — the system must work without it

§13 Build Checklist — Physical Deliverables

This is the list of things the team must have after 3 days. Items marked OPTIONAL should only be attempted if the core flow is fully working.

Deliverable	Priority
React dashboard — case input, Trace button, loading states, error states	MUST
FastAPI backend — /trace endpoint, /health endpoint, /reports/{id} endpoint	MUST
Etherscan API working — fetch + normalise ETH transactions	MUST
Normalised transaction data — unified schema across all API responses	MUST
NetworkX DiGraph — wallet nodes, transaction edges, attributes	MUST
BFS multi-hop tracing — up to 3 hops from suspect wallet	MUST
VASP dataset (vasp_labels.json) — 30+ verified records with source + date	MUST
FIFO fund attribution — traceable_amount and unclassified_amount on every edge	MUST
VASP match output — entity name, type, confidence, source, source_url	MUST
Evidence panel in UI — tx_hash, amounts, timestamps, explorer links, confidence	MUST
ReportLab PDF report — downloadable from UI, contains all evidence + limitations	MUST
JSON cache / fallback — eth_cache.json tested and working for demo reliability	MUST
Multiple-case structure — each case has independent case_id and report	MUST
Real demo data — public, verifiable Ethereum case with known VASP destination	MUST
Unit tests for FIFO — at least 5 edge cases tested before Day 2 integration	MUST
Screenshots — at least one screenshot of every major UI state	MUST
Video backup — clean demo run recorded as insurance against live failures	MUST
TronScan / TRON chain support	OPTIONAL
Heuristic risk flags (rapid forwarding, consolidation pattern)	OPTIONAL
LLM-generated readable summary from verified JSON	OPTIONAL
INR conversion display alongside ETH amounts	OPTIONAL

§14 Debugging & Integration Order

Do not build everything independently and integrate at the end. Each stage must pass before the next stage depends on it. When something breaks, fix it before moving forward.

#	Stage	Owner	Exit Condition	Depends On
1	API works	M1	Etherscan returns a JSON response for a test wallet	None

2	Raw data received	M1	Full transaction list returned and printed to console	Step 1
3	Normalisation works	M1	Raw JSON → unified schema with all required fields	Step 2
4	Graph works	M2	DiGraph created from test transaction list; nodes and edges correct	Step 3
5	Hop tracing works	M2	BFS returns correct path up to 3 hops from start	Step 4
6	FIFO attribution works	M2	traceable + unclassified amounts match \$5 work example	Step 5
7	VASP matching works	M2	Known address returns entity + confidence; unknown returns UNKNOWN	Step 6
8	Backend endpoint works	M2	POST /trace returns complete JSON matching \$5 work example	Step 7
9	Frontend displays results	M3	Real Cytoscape graph rendered from live /trace endpoint	Step 8
10	Evidence works	M4	Evidence table shows all fields; explorer links open correctly	Step 9
11	PDF works	M4	PDF generates, downloads, and contains correct evidence data	Step 10
12	Cache works	M1/M4	Demo runs correctly from eth_cache.json with live API disabled	Step 11
13	Full demo works	All	5 consecutive clean demo runs from cached data	Step 12

Integration rule: M3 builds the UI against mock JSON from Day 1. On Day 2, replace mock with live API call. If the live response doesn't match the agreed schema, fix the schema mismatch immediately — do not patch around it.

§15 Final Team Summary — One Page

Every team member should be able to answer all questions in this section without looking at any other document.

WHAT WE BUILD	A public blockchain investigation workflow for cybercrime investigators. NOT a miniature Chainalysis. A narrow, reliable, demonstrable MVP that converts a suspect wallet address into traceable fund-flow evidence and a structured PDF report.
INPUT	One or more suspect cryptocurrency wallet addresses, provided by the investigator. Optionally: case name, date range, maximum hop depth. CryptoTrace starts here — not at the victim complaint, not at the bank.
PROCESS	Etherscan API → normalise to unified schema → NetworkX directed graph → BFS multi-hop traversal (max 3 hops) → FIFO fund attribution → VASP exact-address matching → evidence assembly → ReportLab PDF generation.
OUTPUT	Interactive fund-flow graph (Cytoscape.js) + evidence table (tx_hash, amounts, timestamps, explorer links, confidence) + downloadable PDF investigation report.
TECHNOLOGY	React + Vite + TypeScript + Cytoscape.js (Frontend) • Python + FastAPI + NetworkX (Backend) • Etherscan API / JSON cache (Data) • ReportLab (Report) • JSON / SQLite / MongoDB (Storage) • Git + GitHub (Version control)
HARD LIMIT	CryptoTrace produces an investigative lead, NOT proof of identity or guilt. It cannot identify the real person behind an exchange account. It does not access bank, UPI, NPCI, or private KYC data. It does not freeze funds automatically. The investigator uses the report to initiate the appropriate legal/compliance process.

3-DAY GOAL

A stable, demonstrable MVP that runs 5 consecutive clean demo runs from cached data. A working PDF report. A complete judge Q&A; defence. A video backup. NOT a large production system. NOT every feature from the problem statement. The strongest student system is honest, narrow, and demonstrable.

The 10-Second Explanation to a Judge

"We start with the wallet a victim reports. Instead of making an investigator trace every transaction manually, our system automatically builds the fund-flow graph, finds a supported VASP endpoint when the evidence allows it, shows why we believe that result, and generates the investigation evidence in one workflow."

Document notes: This is an internal build guide. It is NOT an official I4C document and NOT legal advice. All claims, API endpoints, and VASP records must be verified against current documentation before the demo. Any statement about legal procedure, evidence admissibility, or exchange cooperation must be confirmed by the team. If a judge asks something you cannot prove, say so — a transparent limitation is stronger than an invented answer.

Generated: 30 August 2026 • SIH26183 • CryptoTrace I4C